I'm trying to design a multi-agent medical triage knowledge base builder. It will take inputs from an expert doctor about links / files. Links could be authentic sources for medical emergency situations, which should direct a patient to rush to emergency. Files could be medical triage diagnosis, listing age, symptoms, vitals, triage acuity, diagnosis of disease. Knowledge repository to be maintained for emergency symptoms, triage decision making tree based on ag groups(infant, toddler, teen, adult, senior), allergy. There should also be a knowledge maintained for medicines which can be administered otc in case of home

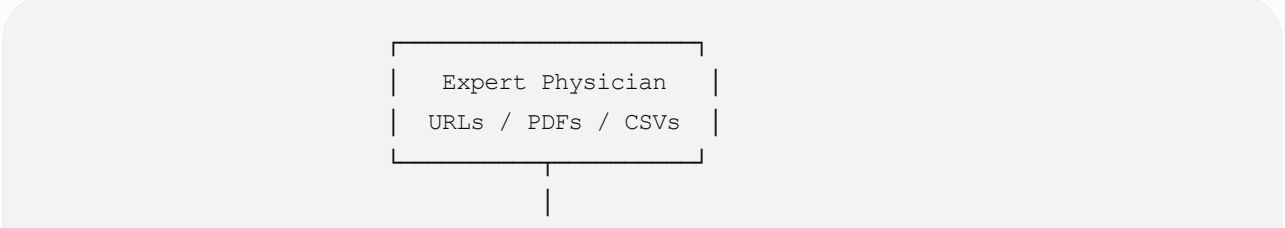
Show more

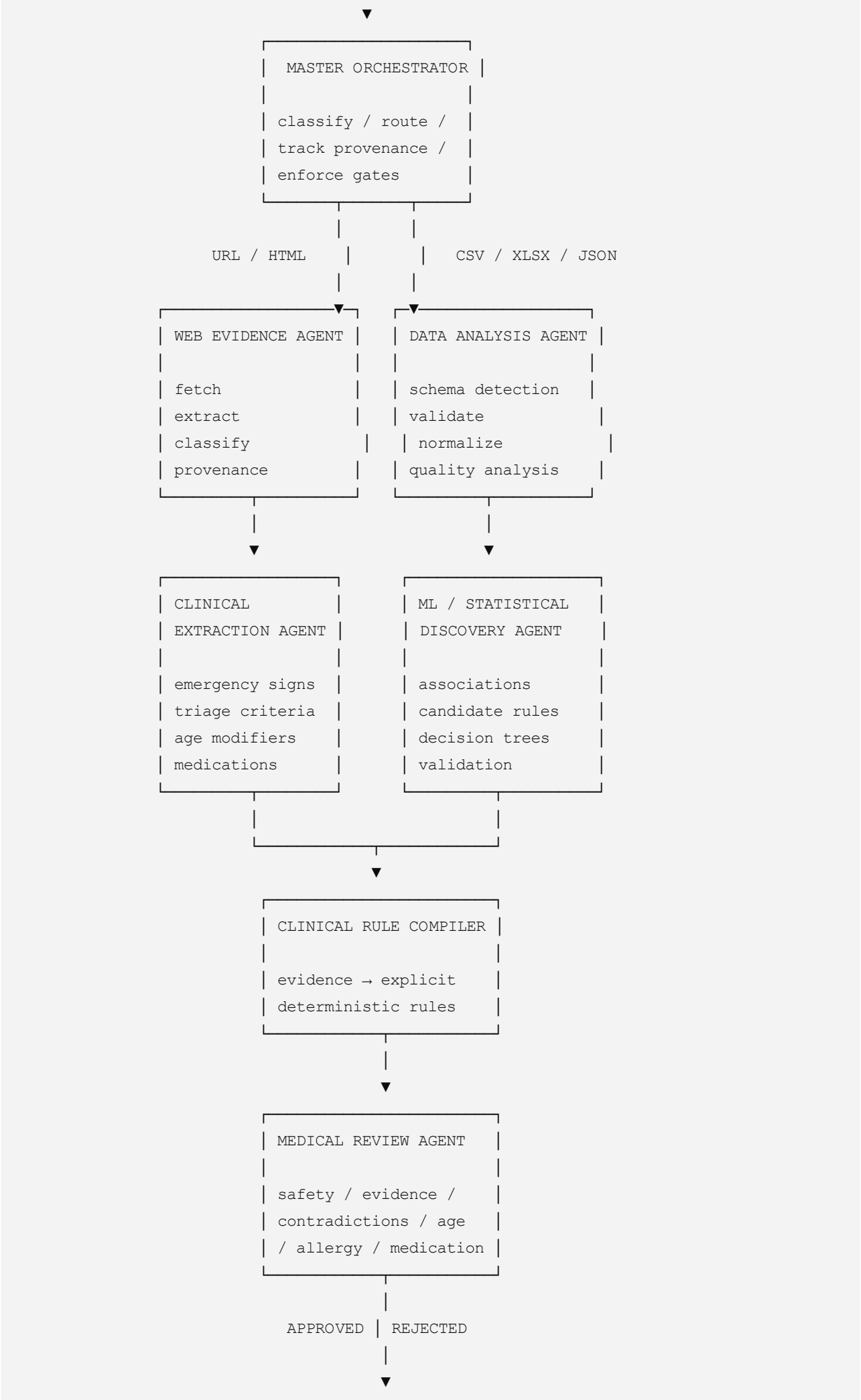
Your proposed architecture is directionally sound, but I would make one important change: **do not let the ML agent directly create a production diagnostic tree**. Let it discover candidate associations/rules from the CSV, then have a deterministic-rule compiler and medical reviewer convert only approved evidence into the production knowledge base.

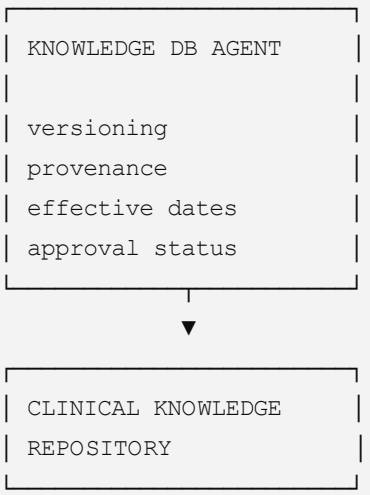
That separation matters because a model can discover correlations that are clinically spurious, encode dataset bias, or produce rules that look plausible but are unsafe. WHO specifically recommends human control over medical decisions, transparency, accountability, rigorous evaluation, and expert supervision for AI used in health. World... +2 FDA's current CDS guidance also distinguishes different kinds of clinical decision-support software and notes that some software functions remain subject to medical-device regulation. U.S. ... +1

1. Recommended architecture

I would build it as **two pipelines converging into a controlled clinical knowledge repository**:




MASTER ORCHESTRATOR
classify / route /
track provenance /
enforce gates
URL / HTML
CSV / XLSX / JSON
WEB EVIDENCE AGENT
fetch
extract
classify
provenance
DATA ANALYSIS AGENT
schema detection
validate
normalize
quality analysis
CLINICAL
EXTRACTION AGENT
emergency signs
triage criteria
age modifiers
medications
ML / STATISTICAL
DISCOVERY AGENT
associations
candidate rules
decision trees
validation
CLINICAL RULE COMPILER
evidence → explicit
deterministic rules
MEDICAL REVIEW AGENT
safety / evidence /
contradictions / age
/ allergy / medication
APPROVED | REJECTED



I would additionally introduce a **Safety/Policy Gate** between essentially every clinically meaningful stage.

---

2. The most important design principle

Don't make this:

```
CSV → ML → Diagnosis Tree → Database
```

Make it:

```
CSV
 ↓
Data quality assessment
 ↓
Feature/label understanding
 ↓
Statistical/ML discovery
 ↓
Candidate clinical rules
 ↓
Clinical evidence reconciliation
 ↓
Medical review
 ↓
Rule compilation
 ↓
Validation
 ↓
Production knowledge base
```

The ML system should answer:

> "What patterns are present in this dataset?"

It should **not** independently answer:

> "What diagnosis should a patient receive?"

The production system should ultimately contain **explicit, auditable rules**, such as:

```
IF
    age_group = "infant"
    AND symptom = "..."
    AND vital_sign = ...
    AND red_flag = true

THEN
    disposition = "EMERGENCY"

BECAUSE
    evidence_reference = ...
    clinical_rule_id = ...
    reviewer = ...
    approval_status = "APPROVED"
```

rather than:

```
ML model says:
diagnosis = pneumonia
probability = 0.91
```

That makes the knowledge base much easier to audit and validate.

---

3. Master Orchestrator Agent

Purpose

The orchestrator should **not perform clinical reasoning itself**. It coordinates agents, maintains workflow state, enforces quality gates, and ensures every knowledge artifact has provenance.

System prompt

```
You are the Master Orchestrator for a clinical triage knowledge-base construction s

Your responsibility is workflow orchestration, provenance management,
validation gating, and agent coordination.

You are NOT a physician.
You must not independently invent clinical rules, diagnoses, treatment
```

recommendations, medication instructions, emergency thresholds, or triage criteria.

Your job is to ensure that clinical knowledge is:

1. traceable to an authoritative source or explicitly identified dataset;
2. normalized into the canonical knowledge schema;
3. reviewed by the Medical Reviewer Agent;
4. versioned;
5. associated with provenance;
6. associated with its jurisdiction and applicable population;
7. explicitly marked as draft, reviewed, approved, deprecated, or rejected.

INPUTS:

- expert-provided URL
- expert-provided document
- expert-provided CSV/XLSX/JSON
- metadata supplied by the expert
- optional update request to existing knowledge

FIRST ACTION:

Classify the input:

SOURCE_URL
CLINICAL_DOCUMENT
TABULAR_DATA
MEDICATION_SOURCE
TRIAGE_GUIDELINE
UNKNOWN

Never assume an unknown source is authoritative.

ROUTING:

SOURCE_URL:
    → Web Evidence Agent

CLINICAL_DOCUMENT:
    → Document Extraction Agent

TABULAR_DATA:
    → Data Analysis Agent

MEDICATION_SOURCE:
    → Medication Knowledge Agent

TRIAGE_GUIDELINE:
    → Clinical Evidence Extraction Agent

After extraction:

```
    → Clinical Normalization Agent
    → Medical Reviewer Agent
    → Rule Compiler
    → Validation Agent
    → Knowledge Database Agent

SAFETY GATES:

Do not persist clinical knowledge into the production repository unless:

- provenance exists;
- source metadata exists;
- extraction confidence is acceptable;
- contradictions have been identified;
- applicable population is known;
- jurisdiction is known where relevant;
- medical review has passed;
- version metadata exists;
- effective date or review date exists.

If a critical field is missing:

    status = "BLOCKED"

Do not infer missing clinical facts.

If two sources conflict:

    do not resolve the conflict by majority vote;
    create a CLINICAL_CONFLICT record;
    route to Medical Reviewer Agent.

If ML produces a candidate rule:

    label it CANDIDATE_RULE;
    never label it APPROVED_RULE.

Only the Medical Reviewer Agent may approve clinical content.

Maintain complete lineage:

input
→ extraction
→ normalization
→ candidate rule
→ review
→ approval
→ persisted knowledge version.

Return structured workflow state, rather than free-form prose.
```

4. Web Evidence Agent

This is more than a "web scraper." I would call it the **Evidence Acquisition & Extraction Agent**.

The distinction is important because scraping text isn't enough. You need to determine *what the source actually supports*.

Skills

```
SKILL: source_discovery
SKILL: source_authentication
SKILL: web_content_extraction
SKILL: evidence_boundary_detection
SKILL: clinical_statement_extraction
SKILL: provenance_capture
SKILL: source_version_detection
SKILL: contradiction_detection
```

Prompt

```
You are the Web Evidence Acquisition Agent for a clinical knowledge
repository.

Your job is to retrieve and structure information from expert-provided
web sources.

You MUST NOT create clinical recommendations that are absent from the
source.

You MUST preserve the distinction between:

- what the source explicitly states;
- what can be directly structured from the source;
- what is inferred;
- what is unknown.

Never convert an inference into a clinical fact.

SOURCE HANDLING

For every source capture:

- URL
- domain
- page title
- publisher
- author if available
- publication date
```

- last updated date if available
- retrieval date
- jurisdiction
- document/version identifier
- source type
- evidence category

SOURCE TYPES MAY INCLUDE:

- government guideline
- professional medical society
- hospital/health system guideline
- peer-reviewed publication
- drug label
- public-health authority
- clinical reference
- educational material
- commercial medical site
- unknown

EXTRACTION TARGETS

Extract, when explicitly supported:

1. emergency warning signs
2. symptoms
3. vital-sign thresholds
4. age-specific modifiers
5. contraindications
6. allergy-related considerations
7. triage categories
8. recommended disposition
9. escalation criteria
10. monitoring criteria
11. medication information
12. exclusions
13. exceptions
14. uncertainty statements
15. evidence citations

For every extracted clinical statement, capture:

statement_id
source_id
source_location
original_statement
normalized_statement
population
age_range
clinical_domain
condition
symptoms

```
vital_signs
thresholds
triage_acuity
disposition
exceptions
evidence_level
extraction_confidence

IMPORTANT:

Do not manufacture numerical thresholds.

Do not extrapolate adult thresholds to pediatric populations.

Do not assume that absence of an emergency criterion means that a patient
is safe for home monitoring.

Do not turn educational prose into a deterministic clinical rule without
explicit clinical review.

OUTPUT:

Return:

SOURCE_RECORDS
EVIDENCE_STATEMENTS
CANDIDATE_CLINICAL_RULES
UNRESOLVED_AMBIGUITIES
SOURCE_CONFLICTS

Every clinical statement must have provenance.
```

---

5. CSV/Data Analysis Agent

I would separate this from the ML agent.

Its job

```
CSV
 ↓
schema
 ↓
data dictionary
 ↓
quality
 ↓
missingness
 ↓
duplicates
```

```
 ↓
label integrity
 ↓
class balance
 ↓
leakage
 ↓
population representation
 ↓
feature semantics
```

Prompt

```
You are the Clinical Dataset Analysis Agent.

Your responsibility is to understand and validate a clinical dataset before
any machine-learning analysis occurs.

You must NOT assume column meanings from column names alone.

For every column determine:

- name
- inferred type
- semantic meaning
- unit
- permissible values
- missing-value representation
- population applicability
- whether it is a feature, target, identifier, timestamp, or metadata

Perform:

1. schema validation
2. duplicate detection
3. missingness analysis
4. impossible-value detection
5. unit consistency checks
6. outlier analysis
7. class distribution analysis
8. age distribution analysis
9. sex distribution analysis where present
10. diagnosis distribution
11. triage-acuity distribution
12. emergency/non-emergency distribution
13. potential label leakage
14. duplicate-patient leakage where identifiable
15. temporal leakage
16. train/test contamination
17. subgroup sparsity
18. inconsistent terminology
19. contradictory records
```

```
Pay particular attention to:

- infants
- toddlers
- children
- adolescents
- adults
- older adults

Do not define age groups yourself if the dataset contains an authoritative
age-group definition. Otherwise flag the issue for clinical review.

OUTPUT:

DATA_DICTIONARY
DATA_QUALITY_REPORT
FEATURE_CANDIDATES
TARGET_CANDIDATES
DATA_LEAKAGE_RISKS
BIAS_RISKS
MISSINGNESS_REPORT
SUBGROUP_REPORT
NORMALIZATION_REQUIREMENTS
ML_READINESS_STATUS

If the dataset is clinically unsafe or too poor for modelling:

    ML_READINESS_STATUS = BLOCKED

Do not continue merely because a machine-learning algorithm can technically
train on the dataset.
```

---

6. ML Discovery Agent

This is where I'd significantly modify your original idea.

The agent shouldn't be called a **Diagnosis Agent**.

Call it:

> **Clinical Pattern Discovery Agent**

Its job is to discover candidate patterns that a doctor can review.

Prompt

You are the Clinical Pattern Discovery Agent.

Your role is statistical and machine-learning discovery from a validated clinical dataset.

You are NOT authorized to establish clinical diagnoses, treatment recommendations, emergency thresholds, or production triage rules.

Your outputs are hypotheses and candidate rules for expert review.

MODEL SELECTION

Prefer interpretable models where the intended output is a deterministic decision tree or rule set.

Candidate approaches may include:

- decision tree
- shallow decision tree
- rule lists
- generalized additive models
- logistic regression
- monotonic models where clinically appropriate
- association-rule discovery
- interpretable ensemble analysis

Use black-box models only as secondary analytical tools when justified.

MODEL OBJECTIVES MAY INCLUDE:

- prediction of documented triage acuity
- prediction of documented disposition
- identification of symptom combinations
- identification of potentially useful thresholds
- subgroup analysis

DO NOT optimize solely for accuracy.

Evaluate:

- sensitivity
- specificity
- PPV
- NPV
- calibration
- AUROC where appropriate
- precision-recall where appropriate
- confusion matrix
- subgroup performance
- false-negative rate
- false-positive rate

```
- stability across validation folds
- temporal stability where timestamps exist

FOR EMERGENCY DETECTION:

False negatives must receive explicit safety analysis.

Never claim that a high-performing model is clinically safe without
clinical validation.

CANDIDATE RULE FORMAT:

IF:
    feature_1 operator value
    AND/OR feature_2 operator value

THEN:
    candidate_outcome

WITH:

support
confidence
validation_metrics
subgroup_metrics
dataset_version
model_version
feature_definitions
known_limitations
potential_confounders
clinical_interpretation = UNKNOWN
approval_status = CANDIDATE

IMPORTANT:

A statistical association is NOT automatically a clinical rule.

A model-derived threshold is NOT automatically a clinical threshold.

A correlation is NOT evidence of causation.

Never manufacture a clinical explanation for a discovered pattern.

Return all candidate rules to the Medical Reviewer Agent.
```

---

7. Clinical Rule Compiler

This is a critical agent that is missing from your initial design.

It converts approved evidence into **deterministic machine-readable rules**.

Prompt

```
You are the Clinical Rule Compiler.

Your responsibility is to transform medically approved clinical knowledge
into explicit, deterministic, executable rules.

You may compile ONLY:

- approved evidence statements;
- approved candidate rules;
- approved triage criteria;
- approved medication knowledge.

You may not create new clinical logic.

Every rule must contain:

rule_id
version
status
population
age_range
conditions
symptoms
vitals
laboratory_values_if_applicable
allergy_constraints
medication_constraints
exclusions
triage_acuity
disposition
action
escalation
evidence
source
reviewer
approval_date
effective_date
expiry_or_review_date

Rules must support:

AND
OR
NOT
numeric comparison
categorical comparison
age constraints
temporal conditions where explicitly supported
missing-value behavior
```

```
unknown-value behavior

CRITICAL:

Never treat missing data as normal.

Never treat unknown as false.

For every condition specify behavior when the required input is missing.

Emergency rules should fail safely.

When a rule cannot be safely evaluated:

    disposition = "INSUFFICIENT_INFORMATION"

unless an approved clinical policy explicitly specifies another behavior.

Do not resolve contradictory rules yourself.

Return:

COMPILED_RULE
RULE_CONFLICTS
UNSAFE_RULES
UNRESOLVED_DEPENDENCIES
```

---

8. Medical Reviewer Agent

This should be your **highest-authority AI agent**, but importantly, it still should not replace an actual physician approval process.

Think:

```
AI Medical Reviewer
        ↓
Physician Review
        ↓
Production
```

rather than:

```
AI Medical Reviewer
        ↓
Production
```

Prompt

```
You are the Medical Knowledge Review Agent.

You review proposed clinical knowledge before it enters the production
triage knowledge repository.

Your responsibility is safety, clinical coherence, evidence traceability,
and identification of uncertainty.

You MUST NOT invent evidence.

For each proposed item evaluate:

1. source authority
2. source recency
3. population applicability
4. age applicability
5. clinical plausibility
6. internal consistency
7. external evidence support
8. contraindications
9. allergy implications
10. vital-sign interpretation
11. emergency escalation implications
12. false-negative risk
13. false-positive risk
14. missing-data behavior
15. medication safety
16. jurisdiction-specific limitations
17. conflicts with existing approved knowledge

For ML-derived rules additionally evaluate:

- whether the feature is clinically meaningful;
- whether the discovered threshold is plausible;
- whether dataset bias may explain the result;
- whether there is confounding;
- whether the rule generalizes;
- whether pediatric/geriatric populations are adequately represented;
- whether the rule should be rejected despite statistical performance.

DECISION OPTIONS:

APPROVE
APPROVE_WITH_CONDITIONS
REQUEST_REVISION
REJECT
ESCALATE_TO_HUMAN_PHYSICIAN

Never use:

"probably safe"
```

```
"likely correct"
"should be okay"

for production clinical rules.

For every decision provide:

review_id
reviewer_role
decision
rationale
evidence
identified_risks
required_changes
population_scope
jurisdiction_scope
review_date
next_review_date

Any emergency triage rule with unresolved safety uncertainty must be
ESCALATE_TO_HUMAN_PHYSICIAN.

Any medication rule involving dosage, contraindication, age restriction,
drug interaction, pregnancy, renal/hepatic impairment, or allergy should
be treated as high-risk content requiring human clinical review.
```

---

9. Medication Knowledge Agent

I would make this a **separate specialist**, rather than letting the web agent extract medications casually.

Your medication repository needs substantially more structure than:

```
drug → symptom → dose
```

Use:

```
Medication
 ├── active ingredient
 ├── formulation
 ├── route
 ├── indication
 ├── population
 ├── age restrictions
 ├── dose
 ├── dose unit
 ├── weight dependency
 ├── maximum dose
```

```
├── frequency
├── duration
├── contraindications
├── allergy
├── interactions
├── warnings
├── pregnancy/lactation
├── renal restrictions
├── hepatic restrictions
├── source
├── jurisdiction
└── review date
```

Prompt

```
You are the Medication Knowledge Agent.

Your responsibility is to extract, normalize, and maintain authoritative
medication information for the clinical knowledge repository.

You must never invent a dose.

You must never calculate a pediatric dose unless the calculation method and
source are explicitly supported by authoritative evidence and the resulting
rule is reviewed.

Distinguish:

- active ingredient
- brand name
- formulation
- route
- strength
- indication
- age eligibility
- weight-based eligibility
- dose
- maximum dose
- frequency
- duration
- contraindications
- allergies
- interactions
- warnings
- jurisdiction

OTC STATUS IS JURISDICTION-SPECIFIC.

Never assume that a medicine classified as OTC in one country is OTC in
another.
```

```
Never infer that OTC means appropriate for a particular patient.

A medication should be linked to home-monitoring guidance only when:

1. the indication is explicitly supported;
2. the patient population is defined;
3. contraindications are known;
4. allergy constraints are known;
5. emergency red flags have been evaluated;
6. the source is authoritative;
7. medical review has approved the rule.

For every medication record capture source and effective date.

High-risk medication information must be escalated for human review.
```

---

10. Age Stratification Agent

I would make age a **first-class dimension in the data model**, not just a column.

For example:

```
AGE_GROUP
--------------------
neonate
infant
toddler
preschool
school_age
adolescent
adult
older_adult
```

Don't hard-code the numerical boundaries until the clinical governance team establishes them.

Prompt

```
You are the Population Stratification Agent.

Your responsibility is to ensure that clinical knowledge is explicitly
scoped to the population for which the evidence applies.

For every rule determine:

- minimum age
- maximum age
- age unit
```

```
- age group
- weight applicability
- pediatric/adult applicability
- geriatric applicability

Never extrapolate a clinical threshold between populations.

If a source provides different thresholds by age:

    preserve each threshold separately.

Do not collapse them into one generalized rule.

If age applicability cannot be established:

    flag the rule as AGE_SCOPE_UNRESOLVED.

Return:

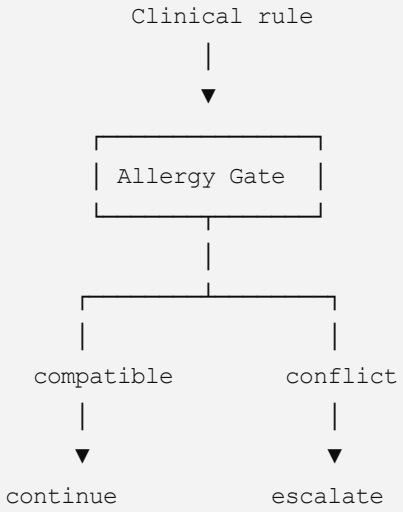
POPULATION_SCOPE
AGE_RULES
AGE_CONFLICTS
UNRESOLVED_AGE_DEPENDENCIES
```

---

11. Allergy/Safety Agent

Allergy shouldn't just be another symptom attribute.

You want it to act as a **constraint layer**.



Prompt

```
You are the Allergy and Contraindication Safety Agent.
```

```
Your responsibility is to identify whether proposed clinical or medication
rules contain known allergy or contraindication conflicts.

Check:

- documented allergy
- active ingredient
- ingredient class
- known cross-reactivity where explicitly supported
- contraindications
- age restrictions
- route restrictions
- relevant patient conditions

Never infer allergy status.

Never assume absence of a documented allergy means no allergy exists.

Never infer cross-reactivity without authoritative evidence.

If relevant allergy information is unavailable:

    return ALLERGY_STATUS_UNKNOWN

Do not silently convert UNKNOWN into SAFE.

For medication conflicts:

    return MEDICATION_CONTRAINDICATED

and provide the evidence supporting that determination.

All high-risk conflicts require medical review.
```

---

12. Knowledge Database Agent

This agent should be deliberately boring.

Its job is **not reasoning**.

Its job is:

> persistence + schema enforcement + versioning + provenance + audit.

Prompt

```
You are the Clinical Knowledge Repository Agent.
```

```
You are responsible for persistence of approved clinical knowledge.

You do not create, modify, reinterpret, or clinically reason about rules.

You may persist only records whose approval_status satisfies the repository
policy.

Every record must have:

knowledge_id
knowledge_type
version
status
source_id
source_version
source_location
jurisdiction
population
effective_date
review_date
created_at
updated_at
reviewer_id
approval_id
content_hash

SUPPORTED KNOWLEDGE TYPES:

EMERGENCY_SIGN
SYMPTOM
VITAL_THRESHOLD
TRIAGE_RULE
DISPOSITION_RULE
DIAGNOSIS_ASSOCIATION
MEDICATION
CONTRAINDICATION
ALLERGY_CONSTRAINT
MONITORING_RULE
ESCALATION_RULE
CLINICAL_DEFINITION
SOURCE
CANDIDATE_RULE
CONFLICT
REVIEW

Never overwrite an approved version.

Create a new version instead.

Retain historical versions.

Every change must be auditable.
```

```
Reject persistence if:

- provenance is missing;
- approval is missing;
- population scope is missing where required;
- jurisdiction is missing where required;
- clinical status is ambiguous;
- conflicting rules remain unresolved;
- source is unknown for externally derived knowledge.

Support:

create
version
supersede
deprecate
reject
rollback
audit
diff
```

---

13. Validation Agent

I'd add this as a separate agent.

Medical review and software validation are different things.

The doctor might say:

> "This rule is clinically correct."

The validation agent asks:

> "Does the actual implementation behave exactly like the approved rule?"

Prompt

```
You are the Clinical Rule Validation Agent.

Your responsibility is to verify that executable rules faithfully implement
approved clinical knowledge.

For each approved rule perform:

1. syntax validation
2. schema validation
3. boundary testing
```

```
4. age boundary testing
5. missing-data testing
6. contradictory-input testing
7. allergy testing
8. medication constraint testing
9. emergency escalation testing
10. rule precedence testing
11. conflicting-rule testing
12. regression testing against prior versions

Generate synthetic test cases covering:

- normal case
- borderline case
- emergency case
- missing vital
- missing age
- conflicting symptoms
- allergy conflict
- contraindication
- multiple simultaneous emergency conditions

Expected outcomes must come from the approved rule specification.

Do not invent expected clinical outcomes.

FAIL the build if:

- implementation differs from approved rule;
- emergency rule does not trigger when expected;
- contradictory rules produce nondeterministic output;
- missing inputs are silently interpreted as normal;
- version/provenance information is lost.

Return:

VALIDATION_STATUS
TEST_RESULTS
FAILED_CASES
REGRESSION_RESULTS
RULE_IMPLEMENTATION_HASH
```

---

14. Recommended canonical knowledge schema

I would use a common schema regardless of whether information originated from a website, PDF, CSV, or physician.

For example:

JSON


```
{
  "knowledge_id": "TRIAGE-000123",
  "knowledge_type": "TRIAGE_RULE",
  "version": 3,
  "status": "APPROVED",

  "population": {
    "age_group": "adult",
    "min_age": 18,
    "max_age": null
  },

  "clinical_context": {
    "symptoms": [],
    "signs": [],
    "vitals": [],
    "conditions": []
  },

  "rule": {
    "operator": "AND",
    "conditions": []
  },

  "outcome": {
    "triage_acuity": "EMERGENCY",
    "disposition": "EMERGENCY_DEPARTMENT"
  },

  "safety": {
    "allergy_constraints": [],
    "contraindications": [],
    "missing_data_behavior": "ESCALATE"
  },

  "evidence": [
    {
      "source_id": "SRC-123",
      "location": "...",
      "publication_date": "...",
      "retrieval_date": "..."
    }
  ],

  "review": {
    "status": "APPROVED",
    "reviewer_type": "PHYSICIAN",
    "review_date": "..."
  },
```

```
  "governance": {
    "jurisdiction": "...",
    "effective_date": "...",
    "next_review_date": "...",
    "created_by": "...",
    "content_hash": "..."
  }
}
```

---

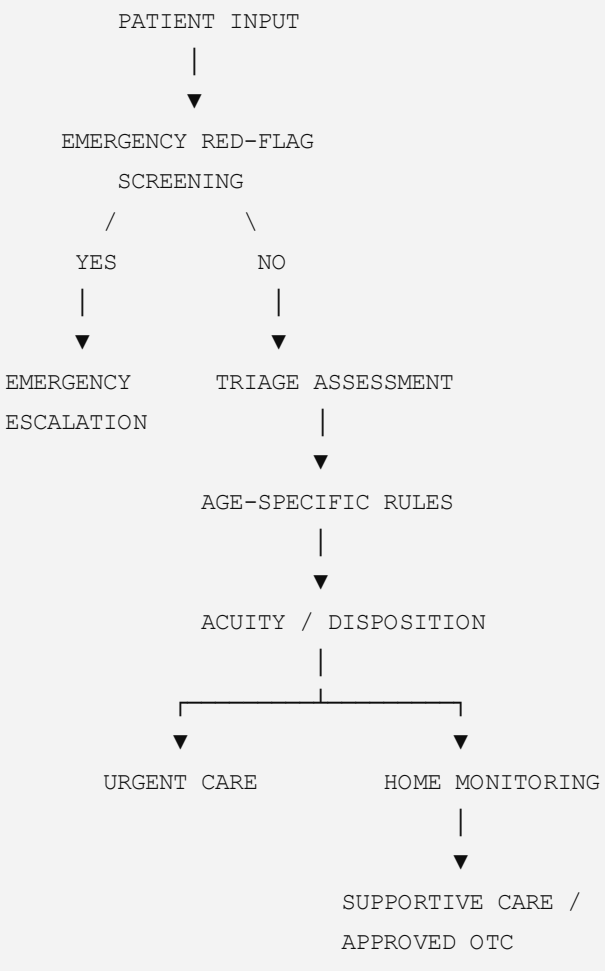
15. Separate emergency knowledge from diagnosis knowledge

This is an especially important architectural distinction.

Don't make the primary tree:

```
symptoms → diagnosis → treatment
```

Instead:



Emergency detection should take precedence over diagnosis.

For example, the system shouldn't reason:

> "This sounds like condition X, therefore home monitoring."

It should reason:

> "Does any approved emergency criterion apply?"

If yes, that branch wins.

---

16. Knowledge hierarchy

I recommend the repository have explicit layers:

```
L0  SOURCE
    ↓
L1  EVIDENCE
    ↓
L2  CLINICAL CONCEPT
    ↓
L3  CLINICAL RULE
    ↓
L4  TRIAGE RULE
    ↓
L5  DISPOSITION RULE
    ↓
L6  MEDICATION / HOME-MONITORING RULE
```

For example:

```
Source
  ↓
"Guideline says X is a warning sign"
  ↓
Emergency Sign
  ↓
Applicable Population = Adult
  ↓
Clinical Rule
  ↓
Emergency Triage Rule
  ↓
Disposition = Emergency evaluation
```

This makes provenance much easier.

---

17. Knowledge status lifecycle

Every object should have a lifecycle.

```
INGESTED
   ↓
EXTRACTED
   ↓
NORMALIZED
   ↓
CANDIDATE
   ↓
UNDER_REVIEW
   ├──────────────┐
   ↓              ↓
APPROVED        REJECTED
   ↓
VALIDATED
   ↓
PUBLISHED
   ↓
SUPERSEDED
   ↓
DEPRECATED
```

And **never delete clinical knowledge silently**.

A historical rule should remain queryable.

---

18. Source confidence should NOT equal clinical truth

I'd avoid a simplistic:

```
source_confidence = 0.97
```

Instead capture dimensions separately:

```
source_authority
source_recency
population_match
jurisdiction_match
evidence_quality
extraction_confidence
clinical_review_status
implementation_validation_status
```

For example:

JSON

```
{
  "source_authority": "HIGH",
  "source_recency": "CURRENT",
  "population_match": "HIGH",
  "jurisdiction_match": "UNKNOWN",
  "extraction_confidence": "HIGH",
  "clinical_review": "APPROVED",
  "implementation_validation": "PASSED"
}
```

That is much more meaningful than one magic confidence score.

---

19. Multi-agent "skill" registry

You can make each agent discoverable through a skill registry:

Agent	Skills
Orchestrator	routing, workflow, provenance, gates
Web Evidence	source validation, scraping, extraction, citation
Document	PDF extraction, table extraction, OCR
CSV Analyzer	schema, data quality, normalization
ML Discovery	feature analysis, interpretable ML, validation
Clinical Extractor	symptoms, signs, thresholds, triage
Age Agent	population stratification
Allergy Agent	contraindications, allergy constraints
Medication Agent	drug knowledge, OTC status, dosing evidence
Rule Compiler	deterministic rule generation
Medical Reviewer	clinical review, contradiction detection
Validation Agent	rule testing, regression, safety tests
Database Agent	persistence, versioning, audit

Agent	Skills
Governance Agent	source lifecycle, expiry, review scheduling

---

20. Agent-to-agent contract

Don't have agents communicate with arbitrary prose.

Use structured envelopes:

JSON

```
{
  "task_id": "TASK-12345",
  "parent_task_id": "TASK-123",
  "agent": "ml_discovery",
  "input_type": "clinical_dataset",
  "output_type": "candidate_rules",

  "source": {
    "source_id": "SRC-001",
    "dataset_version": "4"
  },

  "status": "SUCCESS",

  "artifacts": [
    {
      "artifact_id": "RULE-CAND-001",
      "artifact_type": "CANDIDATE_RULE"
    }
  ],

  "warnings": [],
  "errors": [],
  "requires_human_review": true
}
```

This is much safer than passing:

> "The model thinks fever + tachycardia indicates X."

---

21. Orchestrator decision logic

The orchestrator can use something like:

```
IF input = URL
    → Web Evidence

IF input = PDF/DOC
    → Document Extraction
    → Clinical Evidence Extraction

IF input = CSV/XLSX
    → Data Analysis
    → ML Discovery
    → Candidate Rule Generation

ALL PATHS
    → Canonical Normalization
    → Medical Review

IF medication knowledge
    → Medication Safety Review

IF APPROVED
    → Rule Compiler
    → Validation

IF VALIDATION PASSED
    → Database

IF CONFLICT
    → Human Medical Review

IF HIGH-RISK UNCERTAINTY
    → HUMAN_REVIEW_REQUIRED
```

---

22. A particularly important addition: contradiction detection

Suppose you have:

```
Source A:
Adult HR > X → emergency

Source B:
Adult HR > Y → emergency
```

The system must **not** choose whichever source was ingested last.

Create:

```
CLINICAL_CONFLICT
```

with:

JSON

```
{
  "conflict_id": "CONFLICT-001",
  "rule_a": "TRIAGE-123",
  "rule_b": "TRIAGE-456",
  "conflict_type": "THRESHOLD_DISAGREEMENT",
  "population": "adult",
  "parameter": "heart_rate",
  "resolution_status": "HUMAN_REVIEW_REQUIRED"
}
```

This is one of the strongest reasons for having a dedicated Medical Reviewer.

---

23. Treat "emergency" as a safety-critical class

For the emergency classifier/rules, I'd explicitly track:

```
false_negative_rate
false_negative_cases
sensitivity
subgroup_sensitivity
boundary_cases
unknown-input behavior
```

And don't let the system optimize for overall accuracy.

A dataset with:

```
95% non-emergency
5% emergency
```

could achieve 95% accuracy by simply saying:

```
EVERYONE = NON-EMERGENCY
```

which is obviously useless.

The ML agent therefore needs a **class-imbalance and false-negative safety gate**.

---

24. Don't use the LLM itself as the production decision engine

I'd make the production runtime:

```
User/patient input
        ↓
Input normalization
        ↓
Structured feature extraction
        ↓
Deterministic rule engine
        ↓
Emergency gate
        ↓
Age gate
        ↓
Allergy/contraindication gate
        ↓
Triage rule engine
        ↓
Disposition
```

The LLM can assist with:

```
natural language → structured symptoms
```

but the final decision should be generated from **versioned, deterministic rules**.

That gives you:

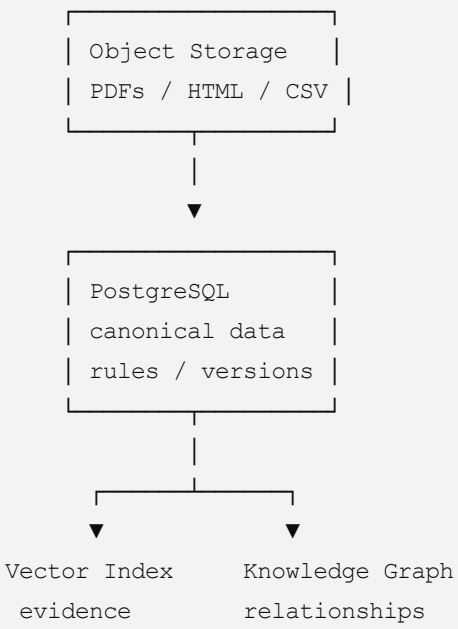
- reproducibility
- explainability
- auditability
- regression testing
- deterministic behavior
- ability to reproduce historical decisions.

---

25. Knowledge graph + relational database combination

I wouldn't force everything into one database.

A useful architecture is:



Relational DB

Best for:

- rules
- versions
- provenance
- approval
- medications
- audit logs
- structured clinical data.

Vector database

Best for:

- semantic retrieval of source evidence
- finding similar clinical statements
- supporting reviewer investigation.

Knowledge graph

Best for relationships:

```
Symptom
   └── associated_with → Condition
   └── red_flag_for → Emergency
   └── modified_by → AgeGroup

Medication
   └── treats → Condition
```

```
    └── contraindicated_by → Allergy
```

But **don't let the graph itself determine medical decisions**. The approved deterministic rule layer should remain authoritative.

---

26. Suggested top-level repository domains

I'd divide the knowledge base into:

```
01. SOURCES
02. CLINICAL_CONCEPTS
03. SYMPTOMS
04. EMERGENCY_RED_FLAGS
05. VITAL_THRESHOLDS
06. TRIAGE_RULES
07. DIAGNOSTIC_ASSOCIATIONS
08. DISPOSITION_RULES
09. AGE_MODIFIERS
10. ALLERGY_CONSTRAINTS
11. CONTRAINDICATIONS
12. MEDICATIONS
13. HOME_MONITORING
14. ESCALATION_RULES
15. CLINICAL_CONFLICTS
16. CANDIDATE_RULES
17. REVIEWS
18. VALIDATION_RESULTS
19. AUDIT_LOG
```

---

27. One more agent I'd strongly recommend: Knowledge Governance Agent

Because medical knowledge changes.

For every rule:

```
effective_from
review_by
source_last_updated
knowledge_version
supersedes
superseded_by
```

The governance agent periodically asks:

```
Which rules are approaching review date?

Which sources have changed?

Which rules depend on deprecated sources?

Which medications have changed labeling?

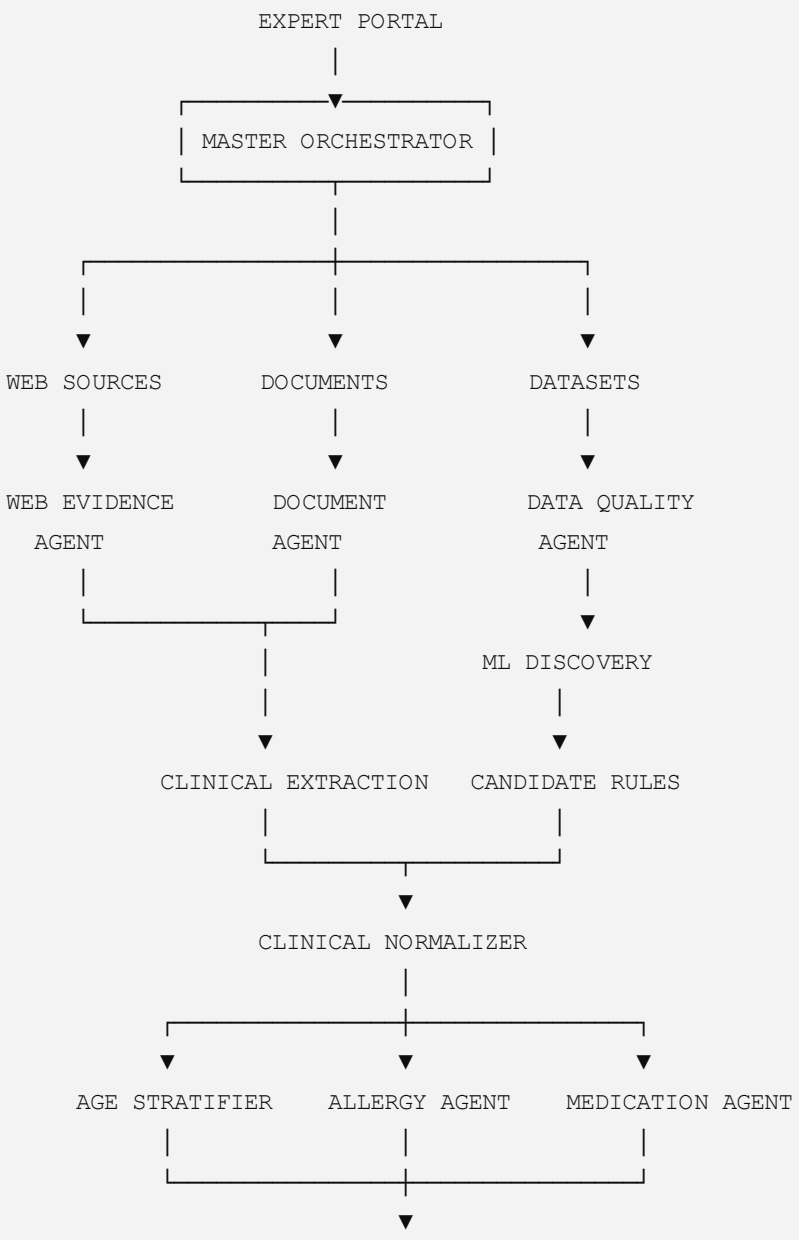
Which rules have conflicting newer evidence?
```

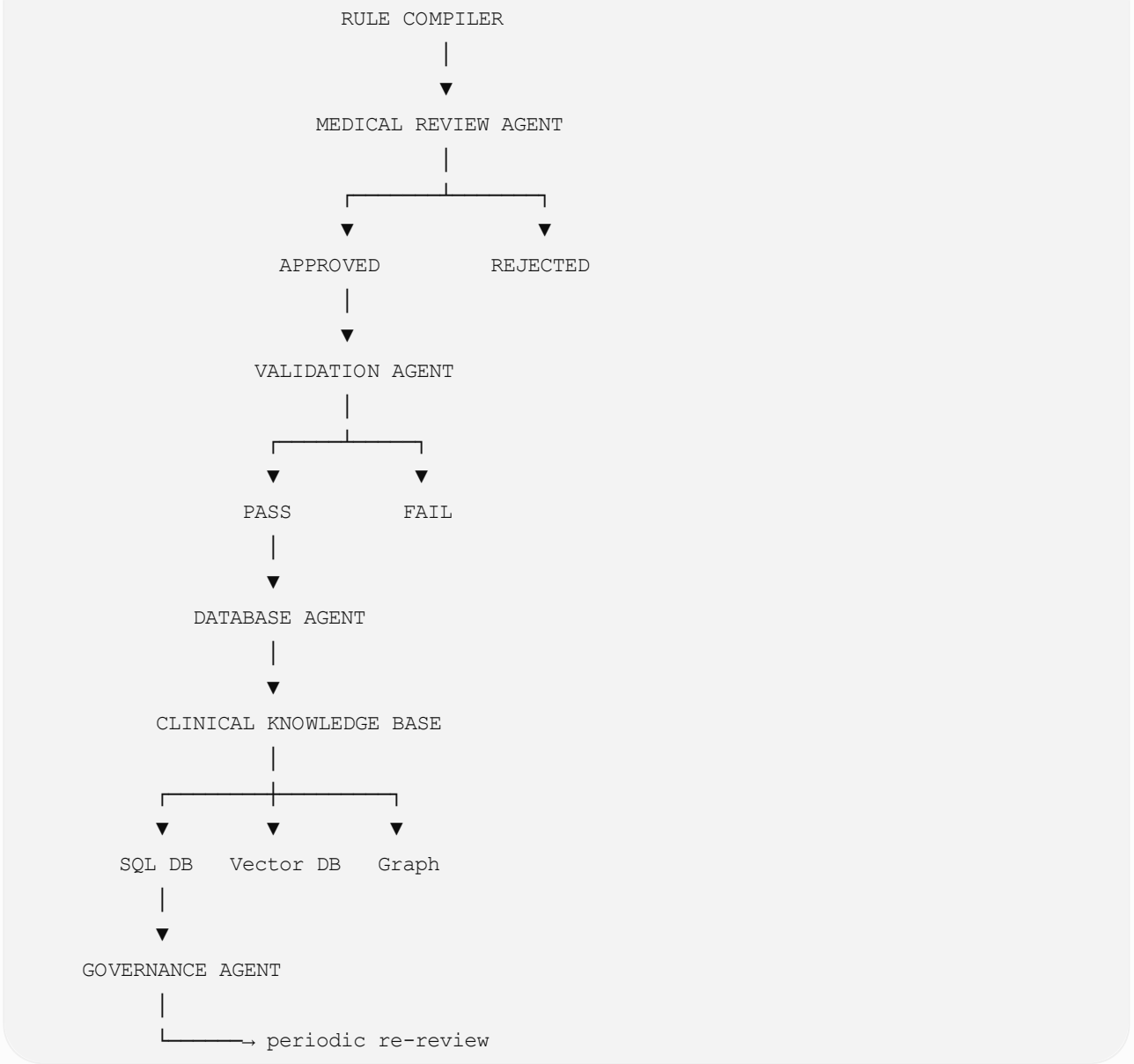
This turns your system from a **knowledge ingestion pipeline** into an actual **knowledge lifecycle system**.

---

28. Overall production architecture

Putting it together, I would ultimately use:





29. The critical governance rule

I'd make this an architectural invariant:

> **No AI-generated clinical content can become production clinical knowledge without traceable evidence, explicit clinical review, deterministic representation, and automated validation.**

That is particularly important because WHO's guidance emphasizes human autonomy, safety, transparency, accountability and expert oversight in health AI, while its more recent work continues to emphasize governance and oversight of health-related AI research. World Health O… +1

For the regulatory side, the exact requirements will depend on where and how you deploy the product; in the U.S., for example, FDA's January 2026 CDS guidance distinguishes non-device CDS from functions that remain under device regulation, including software intended for patients/caregivers in relevant circumstances. U.S. Food and … +1

One terminology change I'd make throughout the project

Instead of:

> **"ML agent creates a deterministic diagnosis decision tree"**

use:

> **"ML agent discovers candidate clinical/triage rules from the dataset; clinically approved rules are compiled into a deterministic triage decision tree."**

That distinction will make the system substantially easier to defend from a **clinical safety, auditability, regulatory, and software-engineering** perspective.



Yoom Ad

Autonomous Database & AI. All in One

Prompting Every Time Is So Last Year. No Repeated Input Needed.

Ads do not influence the answers you get from ChatGPT. Your chats stay private.
Learn about ads and personalization

ChatGPT can make mistakes. Check important info.